# Automating Web Server Configuration on AWS EC2 Using Ansible

Nginx installation, configuration and webpage deployment

Name – Atharva Gavali
PRN - 202301070040

## Executive Summary

This project automates the installation and configuration of an Nginx web server on an Ubuntu Amazon EC2 instance using Ansible. The playbook updates the package cache, ensures Nginx is installed, starts and enables the service, and copies a custom HTML page into the Nginx web root.

The implementation shown in the evidence uses the EC2 Ubuntu machine as both the Ansible control node and the managed target. The inventory uses ansible_connection=local; therefore, the playbook manages localhost rather than a separate remote EC2 host.

### Project Objectives

- Provision and access an Ubuntu EC2 instance on AWS.
- Install and verify Ansible on the Ubuntu machine.
- Define the target host in an Ansible inventory file.
- Create and validate an Ansible playbook for Nginx configuration.
- Deploy a custom HTML webpage through Ansible.
- Verify the Nginx service and access the deployed webpage.
- Publish the project source files and documentation to GitHub.

### Technology Stack

- Amazon Web Services (AWS) – EC2 compute instance
- Ubuntu Linux – operating system
- Ansible – configuration management and automation
- Nginx – HTTP web server
- HTML/CSS – webpage content and presentation
- Git and GitHub – source control and project hosting

# 1. Architecture and Workflow

The EC2 instance hosts Ansible and Nginx. Ansible uses a local inventory connection to apply the configuration to the same Ubuntu machine. After the playbook completes, Nginx serves the HTML file from /var/www/html/index.html over HTTP.

```
Administrator
  |
  v
AWS EC2 (Ubuntu)
  |
  +-- Ansible control node
  |     |
  |     +-- inventory.ini (localhost, local connection)
  |     +-- playbook.yml
  |
  +-- Nginx web server
        |
        +-- /var/www/html/index.html
              |
              v
          HTTP browser access
```

Implementation sequence:

1. Launch the Ubuntu EC2 instance and configure network access.
2. Connect to the instance using SSH / EC2 Instance Connect.
3. Update Ubuntu and verify Ansible availability.
4. Create the project directory, inventory, HTML page and playbook.
5. Run inventory connectivity and playbook syntax checks.
6. Execute the playbook and inspect the recap.
7. Verify Nginx service state and load the webpage in a browser.
8. Commit and push the project files to GitHub.

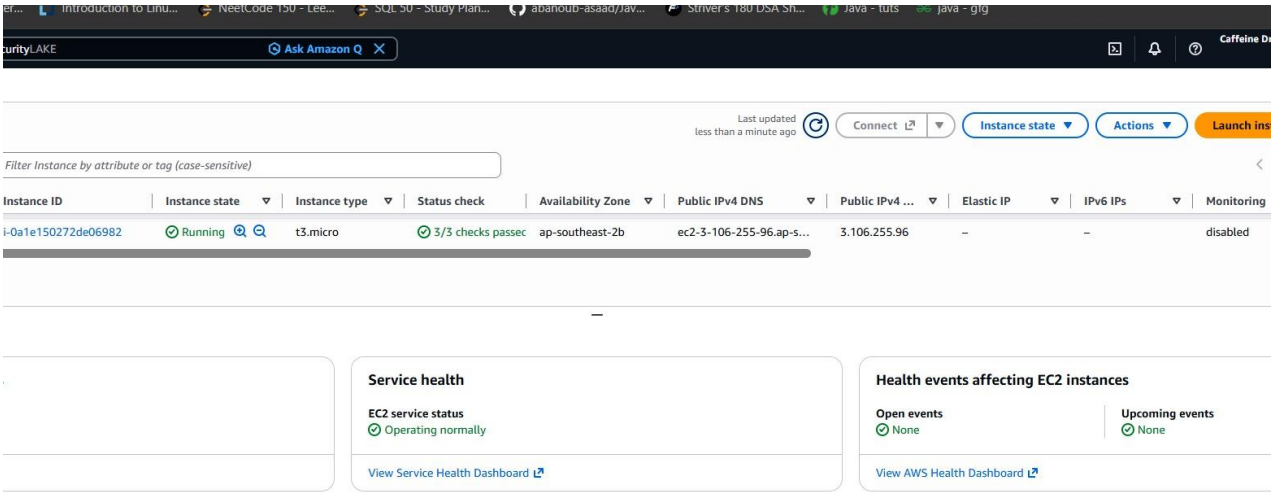



*Figure 1. EC2 console evidence: project instance shown as running.*




*Figure 2. Ubuntu instance terminal session established through EC2 connection.*

# 2. Environment Preparation

The project uses an Ubuntu EC2 instance in the AWS Asia Pacific (Sydney) region. The terminal evidence shows the Ubuntu environment and package-management commands used during preparation.

## 2.1 Update the package index

```
sudo apt update
```

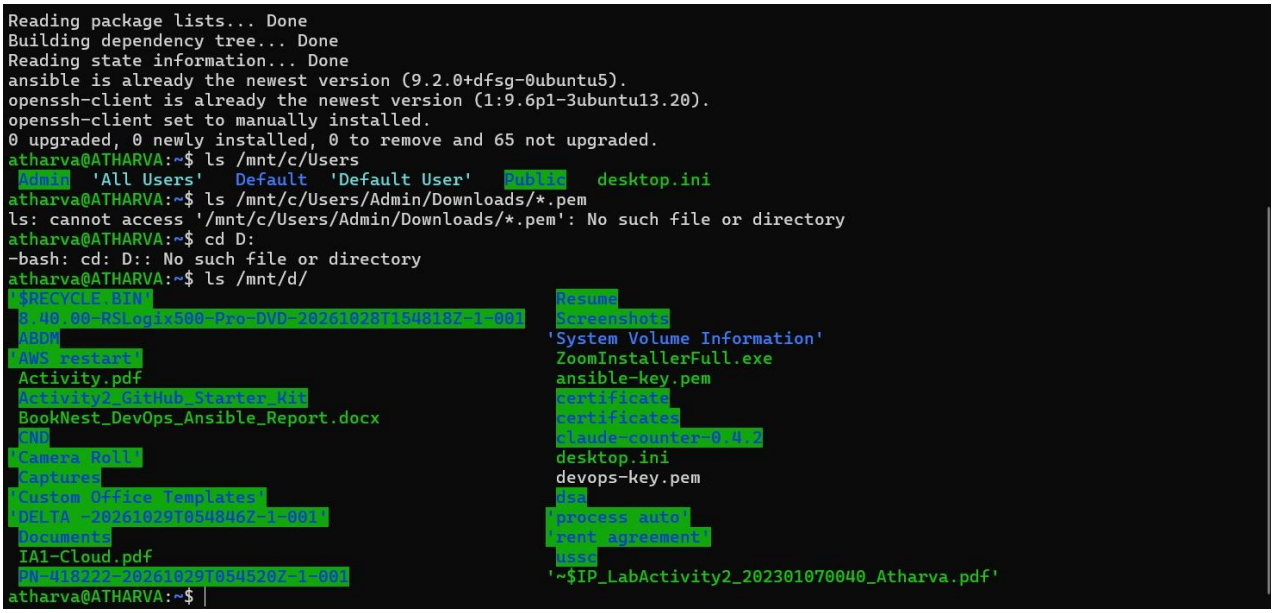



*Figure 3. Package index update output.*

## 2.2 Install / verify Ansible and Git

```
sudo apt install ansible git -y
ansible --version
```

The captured terminal output indicates that Ansible was already installed at the time of the package check. Git was used later to initialize and publish the repository.

```
ansible is already the newest version (9.2.0+dfsg-0ubuntu5).
openssh-client is already the newest version (1:9.6p1-3ubuntu13.20).
openssh-client set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 65 not upgraded.
atharva@ATHARVA:~$ ls /mnt/c/Users
 Admin  'All Users'   Default  'Default User'   Public   desktop.ini
atharva@ATHARVA:~$ ls /mnt/c/Users/Admin/Downloads/*.pem
ls: cannot access '/mnt/c/Users/Admin/Downloads/*.pem': No such file or directory
atharva@ATHARVA:~$ cd D:
-bash: cd: D:: No such file or directory
atharva@ATHARVA:~$ ls /mnt/d/
'$RECYCLE.BIN'                                  Resume
 8.40.00-RSLogix500-Pro-DVD-20261028T154818Z-1-001   Screenshots
 ABDM                                          'System Volume Information'
'AWS restart'                                   ZoomInstallerFull.exe
 Activity.pdf                                   ansible-key.pem
 Activity2_GitHub_Starter_Kit                   certificate
 BookNest_DevOps_Ansible_Report.docx            certificates
 CND                                            claude-counter-0.4.2
'Camera Roll'                                   desktop.ini
 Captures                                       devops-key.pem
'Custom Office Templates'                       dsa
'DELTA -20261029T054846Z-1-001'                'process auto'
 Documents                                     'rent agreement'
 IA1-Cloud.pdf                                  ussc
 PN-418222-20261029T054520Z-1-001              '~$IP_LabActivity2_202301070040_Atharva.pdf'
atharva@ATHARVA:~$ mkdir -p ~/.ssh && cp /mnt/d/ansible-key.pem ~/.ssh/ansible-key.pem && chmod 400 ~/.ssh/ansible-key.p
em && ls -l ~/.ssh/ansible-key.pem
-r-------- 1 atharva atharva 1674 Oct 30 16:55 /home/atharva/.ssh/ansible-key.pem
atharva@ATHARVA:~$
```

*Figure 4. Terminal package output confirming the Ansible installation state.*

# 3. Project Files and Configuration

The repository contains the following implementation files:

```
ansible-devops-project/
├── .gitignore
├── README.md
├── index.html
├── inventory.ini
└── playbook.yml
```

## 3.1 Ansible inventory – inventory.ini

The inventory defines a group named webservers and maps localhost to a local Ansible connection. This matches the single-instance implementation evidenced in the terminal screenshots.

```
[webservers]
localhost ansible_connection=local
```

```
webserver | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
atharva@ATHARVA:~/ansible-devops-project$ nano playbook.yml
atharva@ATHARVA:~/ansible-devops-project$ cat playbook.yml
---
- name: Configure Nginx Web Server
  hosts: webservers
  become: yes

  tasks:
    - name: Update package cache
      apt:
        update_cache: yes

    - name: Install Nginx
      apt:
        name: nginx
        state: present

    - name: Start and enable Nginx
      service:
        name: nginx
        state: started
        enabled: yes
atharva@ATHARVA:~/ansible-devops-project$
```

*Figure 5. Ansible ping module reports SUCCESS for the webservers group.*

## 3.2 Automation playbook – playbook.yml

The playbook targets the webservers inventory group and uses privilege escalation so package and service configuration tasks can run with administrative privileges.

```
---
- name: Configure Nginx Web Server
  hosts: webservers
  become: yes

  tasks:
    - name: Update package cache
      apt:
        update_cache: yes
        cache_valid_time: 3600

    - name: Install Nginx
      apt:
        name: nginx
        state: present

    - name: Start and enable Nginx
      service:
        name: nginx
        state: started
        enabled: yes

    - name: Deploy custom HTML webpage
      copy:
        src: index.html
        dest: /var/www/html/index.html
        owner: www-data
        group: www-data
        mode: '0644'
```

```
- name: Configure Nginx Web Server
  hosts: webservers
  become: yes

  tasks:
    - name: Update package cache
      apt:
        update_cache: yes

    - name: Install Nginx
      apt:
        name: nginx
        state: present

    - name: Start and enable Nginx
      service:
        name: nginx
        state: started
        enabled: yes
    - name: Deploy custom HTML webpage
      copy:
        src: index.html
        dest: /var/www/html/index.html
        owner: www-data
        group: www-data
        mode: '0644'
atharva@ATHARVA:~/ansible-devops-project$ ansible-playbook -i inventory.ini playbook.yml --syntax-check

playbook: playbook.yml
atharva@ATHARVA:~/ansible-devops-project$
```

*Figure 6. Playbook target and task structure.*

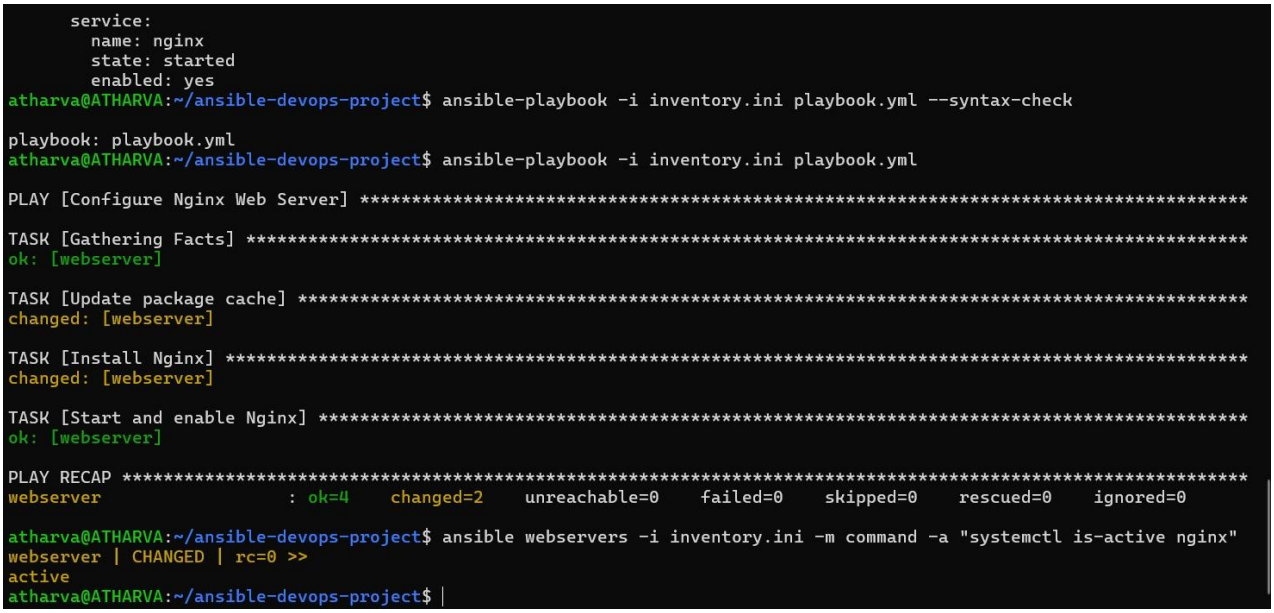



*Figure 7. Service state and webpage copy task in the playbook.*

### 3.3 Webpage – index.html

The index.html file contains the project landing page. The playbook copies this file to /var/www/html/index.html, which is the default document path used by the Nginx configuration in this implementation.

```
<!-- The project HTML file is stored as index.html in the repository. -->
<!-- Ansible copies it to /var/www/html/index.html during deployment. -->
```

## 4. Validation and Execution

### 4.1 Validate inventory connectivity

```
ansible webservers -i inventory.ini -m ping
```

The ping module returned SUCCESS, confirming that Ansible could reach the configured local target.

### 4.2 Check playbook syntax

```
ansible-playbook -i inventory.ini playbook.yml --syntax-check
```

The syntax check returned the playbook name without reporting a YAML or playbook syntax error.

### 4.3 Execute the playbook

```
ansible-playbook -i inventory.ini playbook.yml
```

The execution evidence shows the five playbook stages (including fact gathering) completed. The captured recap reports ok=5, changed=2, unreachable=0 and failed=0 for the successful run.

```
TASK [Deploy custom HTML webpage] ***********************************************************************************
changed: [webserver]

PLAY RECAP **********************************************************************************************************
webserver                  : ok=5    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

atharva@ATHARVA:~/ansible-devops-project$ ansible-playbook -i inventory.ini playbook.yml

PLAY [Configure Nginx Web Server] ***********************************************************************************

TASK [Gathering Facts] **********************************************************************************************
ok: [webserver]

TASK [Update package cache] *****************************************************************************************
changed: [webserver]

TASK [Install Nginx] ************************************************************************************************
ok: [webserver]

TASK [Start and enable Nginx] ***************************************************************************************
ok: [webserver]

TASK [Deploy custom HTML webpage] ***********************************************************************************
ok: [webserver]

PLAY RECAP **********************************************************************************************************
webserver                  : ok=5    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

atharva@ATHARVA:~/ansible-devops-project$
```

*Figure 8. Successful Ansible play recap with zero failed and unreachable tasks.*

# 5. Nginx and Webpage Verification

## 5.1 Verify the Nginx service

```
ansible webservers -i inventory.ini -m command -a "systemctl is-active nginx"
```

The command returned active. This confirms that the Nginx service was running at the time of verification.

```
  (use "git rm --cached <file>..." to unstage)
        new file:   .gitignore
        new file:   index.html
        new file:   inventory.ini
        new file:   playbook.yml

atharva@ATHARVA:~/ansible-devops-project$ git commit -m "Initial commit: AWS Ansible Nginx deployment"
[main (root-commit) 7dbaa88] Initial commit: AWS Ansible Nginx deployment
 4 files changed, 71 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 index.html
 create mode 100644 inventory.ini
 create mode 100644 playbook.yml
atharva@ATHARVA:~/ansible-devops-project$ git log -1 --oneline
7dbaa88 (HEAD -> main) Initial commit: AWS Ansible Nginx deployment
atharva@ATHARVA:~/ansible-devops-project$ git remote add origin https://github.com/atharvag25/ansible-devops-project.git
atharva@ATHARVA:~/ansible-devops-project$ git remote -v
origin  https://github.com/atharvag25/ansible-devops-project.git (fetch)
origin  https://github.com/atharvag25/ansible-devops-project.git (push)
atharva@ATHARVA:~/ansible-devops-project$ git push -u origin main
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 8 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 1.29 KiB | 1.29 MiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/atharvag25/ansible-devops-project.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
atharva@ATHARVA:~/ansible-devops-project$
```

*Figure 9. Nginx service check returns the active state.*

## 5.2 Access the deployed webpage

The webpage was opened in a browser using the EC2 instance public IPv4 address over HTTP. The AWS security group must allow inbound TCP port 80 for external browser access.

```
http://<EC2-PUBLIC-IP>
```

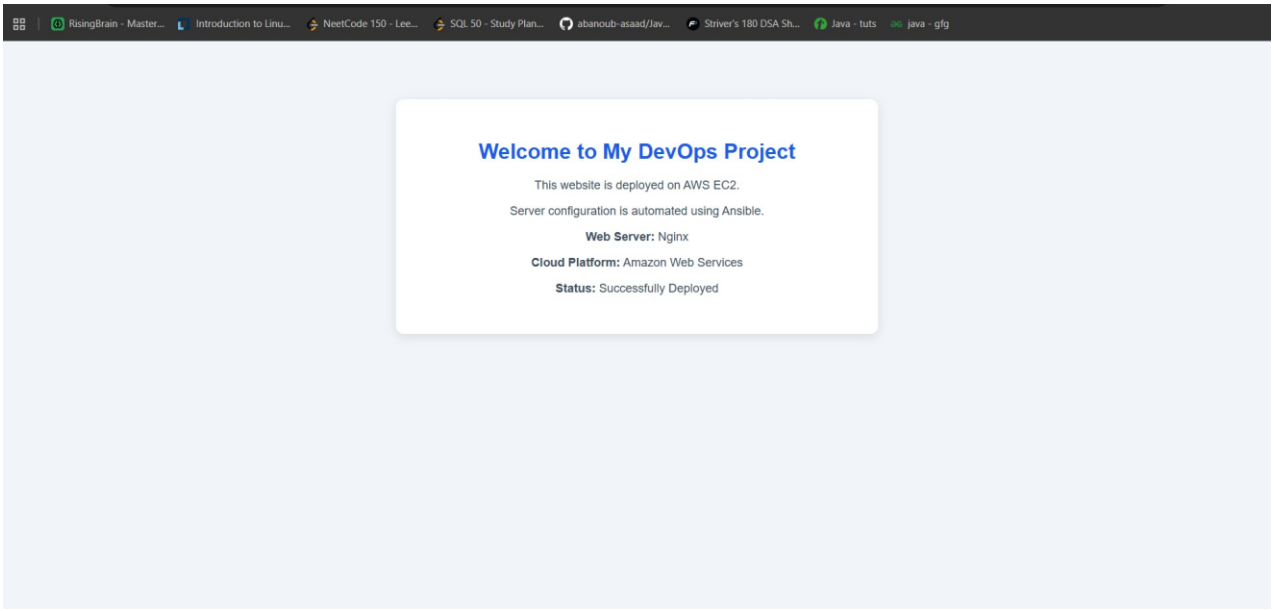



*Figure 10. Browser evidence of the deployed project webpage.*

# 6. Version Control and GitHub Publication

Git was initialized in the project directory. A .gitignore file was added to exclude private key files and local environment artifacts. The project was committed on the main branch and pushed to the public GitHub repository.

```
git init
git branch -M main
git add index.html inventory.ini playbook.yml .gitignore
git commit -m "Initial commit: AWS Ansible Nginx deployment"
git remote add origin https://github.com/atharvag25/ansible-devops-project.git
git push -u origin main
```

Repository URL: https://github.com/atharvag25/ansible-devops-project

```
  (use "git rm --cached <file>..." to unstage)
        new file:   .gitignore
        new file:   index.html
        new file:   inventory.ini
        new file:   playbook.yml

atharva@ATHARVA:~/ansible-devops-project$ git commit -m "Initial commit: AWS Ansible Nginx deployment"
[main (root-commit) 7dbaa88] Initial commit: AWS Ansible Nginx deployment
 4 files changed, 71 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 index.html
 create mode 100644 inventory.ini
 create mode 100644 playbook.yml
atharva@ATHARVA:~/ansible-devops-project$ git log -1 --oneline
7dbaa88 (HEAD -> main) Initial commit: AWS Ansible Nginx deployment
atharva@ATHARVA:~/ansible-devops-project$ git remote add origin https://github.com/atharvag25/ansible-devops-project.git
atharva@ATHARVA:~/ansible-devops-project$ git remote -v
origin  https://github.com/atharvag25/ansible-devops-project.git (fetch)
origin  https://github.com/atharvag25/ansible-devops-project.git (push)
atharva@ATHARVA:~/ansible-devops-project$ git push -u origin main
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 8 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 1.29 KiB | 1.29 MiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/atharvag25/ansible-devops-project.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
atharva@ATHARVA:~/ansible-devops-project$
```

*Figure 11. Git output confirms successful push of main to origin/main.*

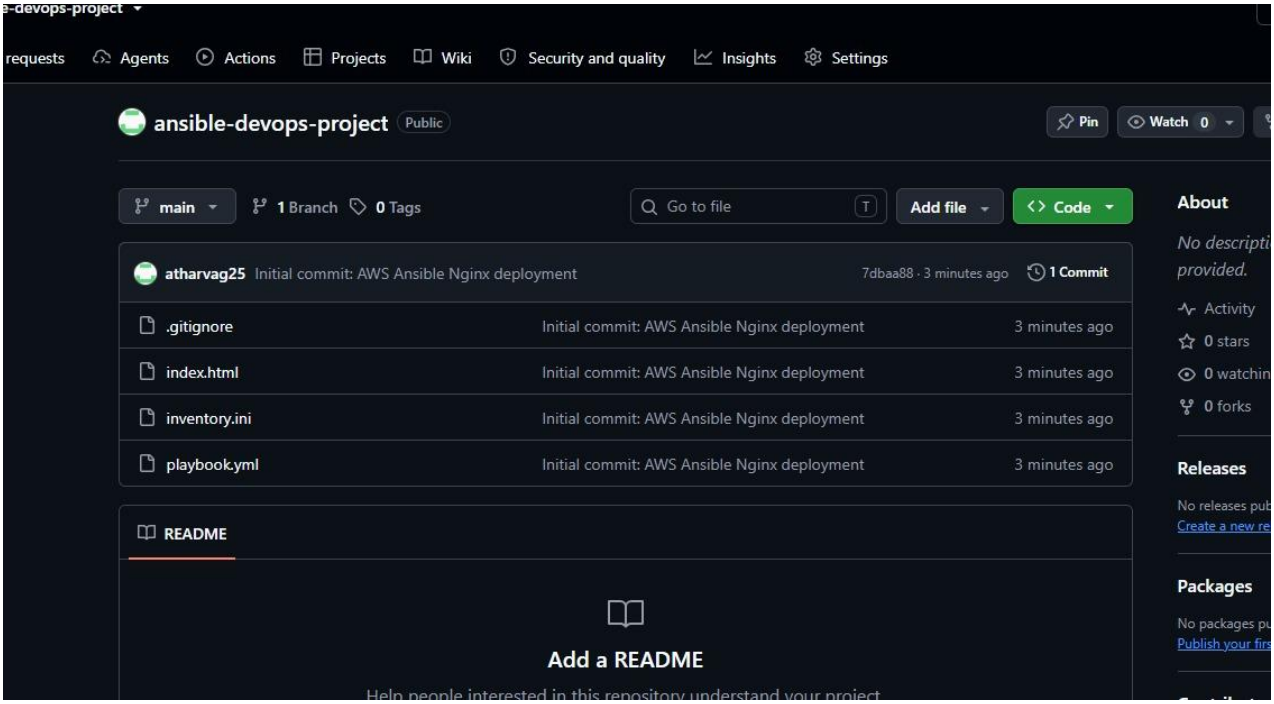



*Figure 12. GitHub repository lists the project files and README.*

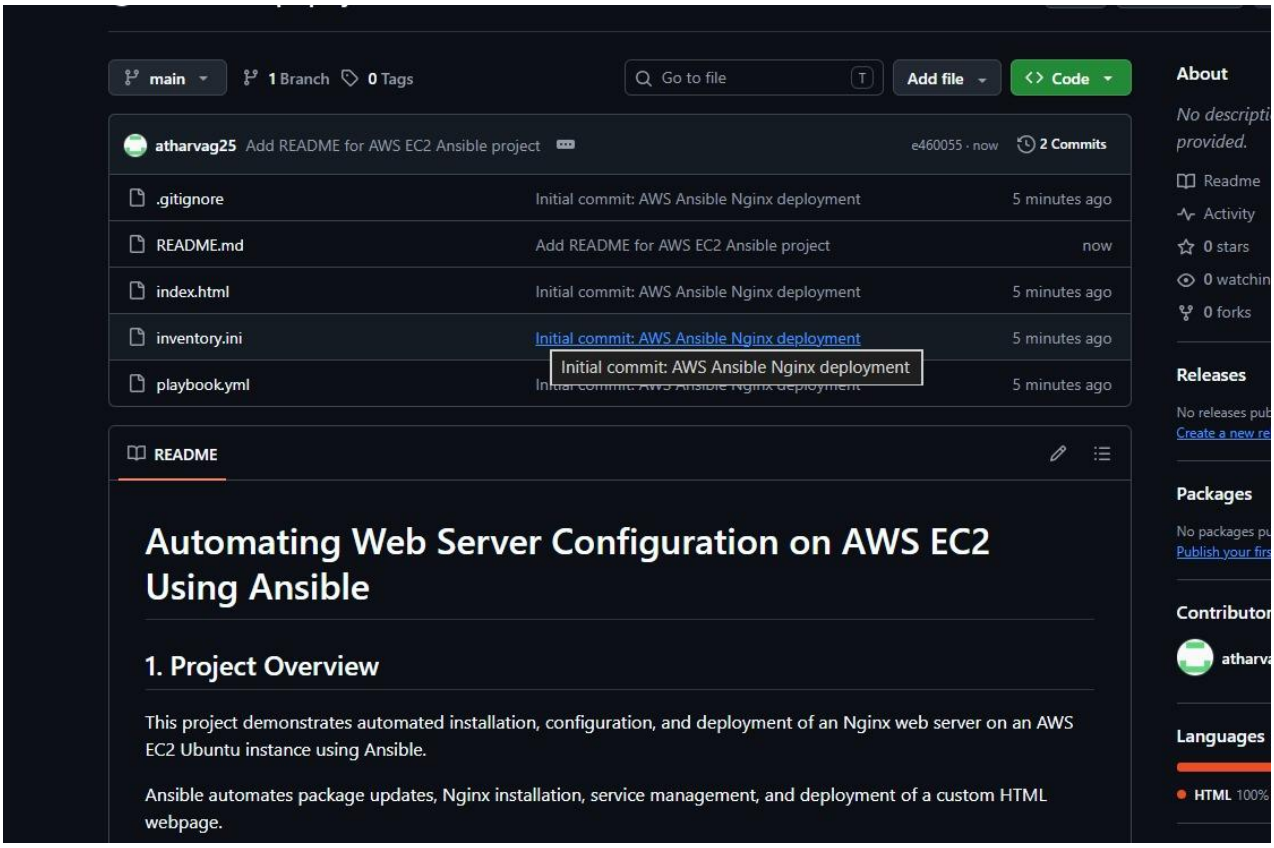



*Figure 13. README is visible on the GitHub repository page.*

# 7. Results

- The Ubuntu EC2 instance was launched and accessed successfully.
- Ansible was available on the Ubuntu environment.

- The inventory connectivity test succeeded.
- The playbook syntax check completed without reported syntax errors.
- The playbook installed/ensured Nginx, started and enabled the service, and deployed the HTML page.
- The Ansible recap showed zero failed and unreachable tasks.
- The Nginx service verification returned active.
- The project files and README were published to GitHub.

# 8. Conclusion

The implementation demonstrates infrastructure configuration automation using Ansible on an AWS EC2 Ubuntu instance. A repeatable playbook was used to prepare Nginx, maintain its service state, and deploy a custom HTML page. Verification was performed through Ansible execution results, a systemd service-state check, and browser access to the hosted page. GitHub provides version-controlled access to the project source and setup documentation.

# 9. Notes and Limitations

- The inventory shown in this project uses localhost with ansible_connection=local. It does not demonstrate Ansible managing a separate remote EC2 worker node.
- The EC2 public IPv4 address can change if the instance is stopped and started unless an Elastic IP is associated.
- For production use, restrict SSH access to trusted source IPs and avoid exposing administrative ports publicly.
- Do not commit private SSH keys (.pem), credentials, or other secrets to GitHub.
- The report uses cropped evidence images to show only the relevant AWS console, terminal, browser, and GitHub content.